What Exact Memory Footprints of GPU Operations Save in Copies and in Unified Memory

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A CUDA program copies its data to the GPU, runs kernels on it, and copies results back. A runtime system that knew exactly which bytes each operation reads and writes, its *footprint*, could leave out of a copy the bytes no later operation reads, and the bytes the GPU has not changed. In graph programs and other irregular code the footprint of a kernel depends on values in memory, so a compiler cannot determine it, and prior systems that use footprints either restrict themselves to regular access patterns or learn them from earlier runs. We measure what exact footprints are worth for copies, using the footprints that the MemPrint hybrid interpreter computes before the program runs on the GPU, and the footprints an NVBit tool records on an NVIDIA A30 as an oracle. We first show that the footprint of one interpreted thread order is not safe to act on. In two GraphBIG programs whose result depends on thread order, the lockstep footprint would leave out bytes that the GPU run needs. We therefore compute the footprint over every thread order the interpreter runs, and count a write as overwriting a copied value only if it happens in every legal order. With these footprints no program loses a needed byte. On 36 small and 37 large benchmarks from Rodinia, GraphBIG and PolyBench-GPU, 22.1% and 4.5% of the bytes copied to the device are never read before they are overwritten. Under a copy cost fitted to the A30, leaving these bytes out speeds up the GPU operations by a geometric mean of 1.018× and 1.056× with the oracle footprints, and saves up to 78.3% of the GPU operation time of `kcore`. With the interpreter's safe footprints, the speedups are 1.008× and 1.052×. The savings come from a few programs, and the gap between the interpreter and the oracle comes from writes that happen only if a loaded value says so, which the interpreter cannot prove happen in every order. We then model the same programs on Unified Memory with less device memory than their data, running each kernel wave by wave of thread blocks. Prefetching the pages each wave touches and evicting the page used farthest ahead speeds the programs up over demand paging by a geometric mean of 1.407× on the 14 large programs that can run in less memory, with the interpreter's footprints exactly as with the traced ones, while prefetching every buffer a kernel uses gains 1.027×.

1 Introduction

GPU programs spend a large share of their time on copies between host and device memory. On the benchmarks of this paper, copies take a median of 23% of the time of all GPU operations on the small inputs and 35% on the large ones. A copy moves a whole buffer, or a range of it, as the programmer wrote it, although the kernels that follow may read only part of what was copied, and a copy back to the host may return bytes that no kernel changed. A runtime system that knew each operation's *footprint*, the bytes it reads and writes, could copy less.

However, the footprint of a kernel in irregular code is chosen by values in memory. A breadth-first search reads the neighbour list of each vertex in its frontier, so the bytes it reads depend on

the graph. Static analyses of kernel indices, such as CODA [4] and LADM [3], compute the data each thread block accesses when its indices follow a stride, and CODA falls back to profiling or to interleaving the data across devices when they do not. Runtime systems for Unified Memory, such as SUV [2], decide placement per data structure, and MSched [6] predicts a kernel's working set from its launch arguments, learned from earlier runs. Therefore, it is not known how much exact footprints of irregular kernels are worth.

The MemPrint hybrid interpreter computes exact footprints before the program runs on the GPU. It runs the program's host code natively and interprets each kernel on the real memory contents, so a loaded value, a branch on it and an address computed from it are known exactly. It runs every kernel under four thread orders, because the footprint of a kernel whose threads race through memory can depend on the order. We measure what its footprints save in copies between host and device, against the footprints that an NVBit [7] tool records on the GPU.

Footprints matter more when the program's data does not fit in device memory. With Unified Memory, the GPU moves pages between host and device as kernels touch them, and every page a kernel touches that is not on the device stops the kernel for a page fault. Irregular programs suffer most from such oversubscription [5]. A runtime system that knew which pages each part of a kernel touches could move them before the kernel needs them, and evict the pages that are needed latest. We model this on the same benchmarks, with the footprints of each wave of thread blocks, and compare it with demand paging and with prefetching every buffer a kernel uses.

In this work, we make the following contributions:

- We show that the footprint of one interpreted thread order is not safe to act on, because in two GraphBIG programs whose result depends on thread order, the lockstep footprint would leave bytes out of a copy that the GPU run needs (Section 3).
- We define the footprint over every interpreted thread order, in which a write overwrites a copied value only if it happens in every legal order, and we compute it both during the interpreter's run and from the references it already records (Section 3).
- We measure, on 36 small and 37 large benchmarks, the bytes of each copy that no operation needs and the GPU time that leaving them out saves, for the traced footprints, the interpreter's footprint over every order, and its lockstep footprint (Section 6.1).
- We model the benchmarks on Unified Memory with less device memory than their data, with kernels run wave by wave of thread blocks, and measure what prefetching and eviction gain over demand paging with the footprints of each wave (Section 5 and Section 6.2).

2 Background

Operations and footprints. A CUDA program issues a sequence of *operations* on the GPU: kernel launches, copies between host and device memory, and memory fills. We describe the footprint of an operation in units of 4 bytes of a device allocation: for each unit, whether the operation reads it and whether it writes it. A copy to the device writes the units it copies, and a copy to the host reads them.

The hybrid interpreter. The hybrid interpreter of MemPrint compiles the program's host code against a stand-in CUDA runtime whose device memory is host memory, and runs it natively. When the host code launches a kernel, the interpreter runs all threads of the kernel at once as arrays, on the real contents of memory. It records, for each operation, the units it reads and writes. A

reference is *certain* if it happens in the run, and *possible* if it may happen, for example through a pointer the interpreter could not place. Threads of a kernel can race through memory, for example when they lower a value with `atomicMin` or claim a slot of a worklist, and the result can then depend on the order in which the GPU runs them. The interpreter therefore runs each program under four thread orders. The first is *lockstep*, which runs all threads of a kernel together. In the other three, each block runs to completion before the next starts, in the order of block numbers, in reverse order, or in a random order. A unit is written *in every legal order* if a write to it is certain, and its address and the conditions under which it happens depend on no value loaded from memory.

The traced footprints. An NVBit tool records the units each operation of a GPU run reads and writes, on an NVIDIA A30. The traced footprints are exact for that run, and we use them as the oracle a footprint analysis cannot improve on.

3 Footprints over every thread order

The interpreter merges the dependences of its four thread orders, but before this work it wrote out the footprint of the lockstep run only. A runtime system that leaves bytes out of a copy needs a footprint that holds for every order, because the order in which the GPU runs a racing kernel is not known in advance. In `bfs-topo-atomic`, threads lower the level of each vertex with `atomicMin`, and which round lowers a level last depends on the order. Its lockstep footprint would leave out 4 bytes of a copy to the device that the traced run reads. In `connected-comp`, the number of rounds changes from run to run, and its lockstep footprint would leave out 4 bytes of a copy to the host that the traced run wrote.

We define the footprint of an operation over every order as follows. First, we match the operations of each order to those of the lockstep run, from both ends of the sequence, as the interpreter does when it merges dependences. Second, a unit is read or written if some order reads or writes it in the matched operation. Third, a unit is *overwritten* only if every order that issues the operation writes it in every legal order, so that a later read sees this operation's value whatever the order. Fourth, an order that runs rounds the lockstep run does not have adds what those rounds touch to the lockstep run's kernels between the same matched operations, as possible references. The interpreter now writes this footprint during its run. For runs made before, we rebuild it from the per-block references that the interpreter already records for each order. On `gemm` and `bfs-topo-atomic`, the two give identical footprints.

4 What footprints save in copies

We count, for each copy, the units it moves that no operation needs, from a footprint of every operation of the run.

Copies to the device. A unit that a copy to the device writes is *unread* if no later operation reads it before an operation overwrites it. A read of a unit and a write to it in one operation read the old value. A copy back to the host counts as a read, so a unit that the program copies in and straight back out is needed.

Copies to the host. A unit that a copy to the host reads is *unwritten* if no device operation wrote it since it was last copied in either direction, so that the host already has its value. This

assumption holds only if the copy returns the unit to the host buffer it came from or went to last, which the footprints do not record, so the unwritten units are an upper bound on what a runtime system could leave out.

Copy cost. We model a copy as a fixed cost plus its bytes over a bandwidth, per direction, from all copies that Nsight Systems profiled in the benchmarks of both input sizes. The fixed cost is the median time of copies of 4 KB or less, and the bandwidth is the median over copies of 1 MB or more. A copy to the device costs 1.0 µs plus 11.9 GB/s, and a copy to the host 1.5 µs plus 9.0 GB/s. A copy that leaves out some units moves each contiguous range of the needed units as a copy of its own, and saves the share of the measured copy time that the model gives, or nothing if the ranges cost more than the whole copy. We report the time saved as a share of the time of all GPU operations of the program, issued one after another.

5 Footprints for Unified Memory

We model each program as if `cudaMallocManaged` allocated its device memory, rather than `cudaMalloc`, so that its device allocations are managed memory. A copy to the device becomes a write by the host, so the pages it writes are on the host afterwards, and a copy to the host becomes a read by the host, which brings back the pages the device holds. Device memory holds a share of the pages that the program's device operations touch, from 25% to 100% in steps of 12.5%, with pages of 64 KB, the basic block in which Unified Memory moves data, or of 4 KB.

Waves of thread blocks. The GPU runs a kernel's blocks as far as they fit at once, so a kernel that touches all of the program's pages may touch only part of them at a time. We run each kernel as a sequence of *waves*. A wave holds as many blocks as the A30 runs at once, in block order, which follows from the threads, registers and shared memory of a block and the 56 streaming multiprocessors of the A30. Each wave takes an equal share of the kernel's profiled time and touches the pages that its blocks touch in the traced run, which the NVBit tool records per block. We simulate a share of memory only if the pages of every wave fit in it, because how a wave thrashes when they do not depends on the order of its blocks' accesses, which the footprints do not record.

Policies. We compare four policies. First, *demand paging* moves a page when a wave touches it and the page is not on the device, and evicts the least recently used page. Second, *buffer prefetching*, as a compiler that knows which data structures a kernel uses could do, moves every page of every buffer the kernel touches before its first wave, as far as they fit beside the wave's own pages, and evicts the least recently used page. Third, *interpreter prefetching* moves, before each wave, the pages that the interpreter's blocks of that wave touch under any of its four thread orders, and evicts the page whose next use in that footprint is farthest away. Fourth, the oracle does the same with the traced footprints. Every policy pays for what the traced run actually touches, so a prediction that is wrong costs faults or moves that are not needed.

Costs. A fault stops the wave, and the faults of a wave are handled in batches of up to 256, each of which costs a fixed time of 10, 25 or 50 µs, plus the transfer of each page at the bandwidth of the copy model of Section 4. A prefetch pays the transfer only. Evicting a page that the device wrote since it arrived copies it back to the host. We report each policy's speedup over demand paging, as a geometric mean over the programs and shares of memory, at 25 µs unless we say otherwise.

Footprints	Small inputs (36 programs)					Large inputs (37 programs)				
	HtoD unread	DtoH unwritten	Speedup	≥1% programs	Unsafe	HtoD unread	DtoH unwritten	Speedup	≥1% programs	Unsafe
Traced	22.1%	1.1%	1.018	12	–	4.5%	2.5%	1.056	9	–
Interpreter, all orders	21.9%	1.1%	1.008	8	0	4.4%	2.5%	1.052	4	0
Interpreter, lockstep	22.1%	1.1%	1.017	12	2	4.5%	2.5%	1.055	9	2

Table 1: Copies between host and device with each footprint: the share of the bytes copied to the device that are unread and of the bytes copied to the host that are unwritten, the geometric-mean speedup of all GPU operations from leaving them out, the programs that save at least 1% of their GPU operation time, and the programs on which the footprint marks a unit as not needed that the traced run needs.

Program	Suite	Small inputs				Large inputs			
		Copies	Traced	All orders	Lockstep	Copies	Traced	All orders	Lockstep
`kcore`	GraphBIG	68	8.0	5.7	8.0	92	78.3	78.3	78.3
`jacobi1d`	PolyBench	14	0.0	0.0	0.0	57	13.3	13.3	13.3
`3mm`	PolyBench	26	9.0	9.0	9.0	29	11.0	11.0	11.0
`bfs`	Rodinia	36	10.7	1.1	10.7	55	5.4	0.5	5.4
`2mm`	PolyBench	30	5.3	5.3	5.3	35	6.0	6.0	6.0
`gc-topo-thread-centric`	GraphBIG	12	4.7	0.1	4.7	7	1.9	0.0	1.9
`bfs-topo-thread-centric`	GraphBIG	28	4.7	1.2	4.7	24	1.6	0.4	1.6
`connected-comp`	GraphBIG	19	4.5	0.0	3.5	24	2.8	0.0	1.5
`sssp-topo-thread-centric`	GraphBIG	31	3.9	1.0	3.9	35	1.6	0.4	1.6
`backprop`	Rodinia	91	2.8	2.8	2.8	98	0.7	0.7	0.7
`betweenness`	GraphBIG	5	2.2	0.6	2.2	–	–	–	–
`bfs-topo-atomic`	GraphBIG	14	2.1	0.0	1.9	10	0.6	0.0	0.5
`bfs-data-thread-centric`	GraphBIG	22	1.3	1.3	1.3	19	0.6	0.6	0.6

Table 2: The programs whose traced footprints save at least 1% of their GPU operation time on either input: the share of that time spent in copies, and the share saved with each footprint (%).

6 Evaluation

Setup. We use the benchmarks, inputs, traces and interpreter runs of MemPrint: 8 programs from Rodinia, 10 from GraphBIG and 20 from PolyBench-GPU, unmodified, on small inputs and on larger ones (GraphBIG on a generated graph of 8,192 vertices). GraphBIG's `betweenness` has no run on the large input. On the small input, PolyBench's `corr` and `covar` launch most of their kernels with no thread block, because the grid size is the input size divided by 256, so their kernels do not run and their copies to the host return what was copied in. We leave both out of the small input. For each program we compare three footprints: the traced footprints, the interpreter's footprint over every order (Section 3), and its lockstep footprint. A footprint is *unsafe* on a program if it marks a unit as not needed that the traced run needs.

6.1 Copies between host and device

Results. Table 1 summarises the results. *With its footprint over every order, the interpreter never leaves out a unit that the GPU run needs, on either input.* Its lockstep footprint is unsafe on 2 programs on each input. The bytes copied to the device that are never read are 22.1% of all such bytes on the small inputs and 4.5% on the large ones, and the interpreter finds almost all of them (21.9% and 4.4%). Leaving out the unneeded units speeds up the GPU operations by a geometric mean of 1.018× and 1.056× with the traced footprints, and by 1.008× and 1.052× with the interpreter's footprint over every order.

					Oversubscribed			All data fits		
Model	Inputs	Page	Programs	Shares	Buffers	Inter-preter	Traced	Buffers	Inter-preter	Traced
Kernels	Large	64 KB	9 of 37	26	0.572	1.294	1.294	1.094	1.114	1.118
Waves	Large	64 KB	18 of 37	60	1.023	1.337	1.337	1.144	1.144	1.148
Waves	Large	4 KB	14 of 37	55	1.027	1.407	1.407	1.192	1.192	1.196
Waves	Small	64 KB	31 of 36	69	1.215	1.323	1.324	1.217	1.223	1.225
Waves	Small	4 KB	8 of 36	30	1.221	2.165	2.165	1.358	1.361	1.363

Table 3: Unified Memory with each policy: the programs and shares of device memory at which every operation (with kernels as whole operations) or every wave fits in less memory than the program's data, and the geometric-mean speedup over demand paging of buffer prefetching and of prefetching with the interpreter's and the traced footprints, over those shares and when all of the data fits, at 25 µs per batch of faults.

Where the savings come from. The savings are concentrated in a few programs (Table 2). In GraphBIG's `kcore`, no operation of the run reads most of the edge array the program copies to the device, so 97% of the bytes copied to the device on the large input are never read, and leaving them out saves 78.3% of its GPU operation time, with the interpreter's footprints as with the traced ones. In PolyBench's `2mm`, `3mm` and `jacobi1d`, the program copies buffers to the device that a kernel overwrites before any kernel reads them. In `jacobi2d` and Rodinia's `nw`, a kernel also overwrites about half of the copied bytes before reading them, but these bytes lie in 998 and 128 separate ranges on the small input, so the fixed cost of copying each needed range between them outweighs the bytes saved, and the model saves nothing.

The cost of safety. The interpreter's footprint over every order saves less than the traced footprints in the GraphBIG programs and in Rodinia's `bfs` (1.1% against 10.7% on the small input). In these programs a kernel writes a vertex's level or flag only if a loaded value says so, so the interpreter cannot prove that the write happens in every legal order, and it keeps the copied value as possibly read. The lockstep footprint reaches the traced savings on these programs, because the lockstep run does write the units, but the same footprint is unsafe on `bfs-topo-atomic` and `connected-comp`, where the order changes which units a kernel writes.

6.2 Unified Memory with less device memory than data

Results. Table 3 summarises the results. *Prefetching with the interpreter's footprints gains as much over demand paging as prefetching with the traced footprints*: on the large inputs with pages of 4 KB, 1.407× for both, and on every input and page size the interpreter falls more than 1% behind the oracle on 1 configuration only, and the oracle never loses to demand paging. Buffer prefetching gains 1.027× on the same configurations. The gain grows with the cost of a fault, from 1.226× at 10 µs to 1.677× at 50 µs. When all of the data fits, prefetching still saves the faults of the first touch of each page, and the three policies are within 1% of each other (1.192×, 1.192× and 1.196×).

Where the gain comes from. The interpreter's prefetching gains in two ways. It moves the pages of a wave before the wave starts, so the wave takes no fault, and it evicts the page needed latest, so it moves fewer pages: 36% fewer bytes than demand paging over all oversubscribed configurations on the large inputs, in 20 of the 55 configurations. Buffer prefetching moves pages that the current wave does not touch, and later waves evict them. In `gramschm`, whose kernels run

Program	Suite	Pages	Waves	Memory	Buffers	Interpreter	Traced
`fdtd-2d`	PolyBench	433	60	87.5%	1.40	2.28	2.28
`3dconv`	PolyBench	4,064	126	25.0%	0.95	2.03	2.03
`3mm`	PolyBench	112	3	50.0%	1.33	1.37	1.37
`jacobi2d`	PolyBench	1,953	360	25.0%	1.00	1.33	1.33
`srad`	Rodinia	1,536	12	50.0%	0.98	1.31	1.31
`pathfinder`	Rodinia	9,864	10	25.0%	1.06	1.31	1.31
`hotspot`	Rodinia	768	5	37.5%	0.89	1.28	1.28
`2dconv`	PolyBench	2,046	10	25.0%	0.91	1.20	1.20
`backprop`	Rodinia	2,308	20	25.0%	0.96	1.18	1.18
`2mm`	PolyBench	320	2	62.5%	1.11	1.11	1.11
`gemm`	PolyBench	768	3	62.5%	0.98	1.07	1.07
`gramschm`	PolyBench	192	768	75.0%	0.65	1.05	1.05
`syrk`	PolyBench	512	3	75.0%	1.01	1.03	1.03
`syr2k`	PolyBench	768	3	87.5%	1.01	1.02	1.02

Table 4: The large programs that run in less memory than their data, with pages of 4 KB: their pages, the waves of all their kernels, the smallest share of the pages that holds every wave, and each policy's speedup over demand paging there.

768 waves, it moves 101 MB to the device where demand paging moves 0.8 MB, and it is slower than demand paging (Table 4).

Waves against whole kernels. With kernels as whole operations, a program can run in less memory only if no single kernel touches all of its pages, and on the large inputs with pages of 64 KB that holds for 9 of the 37 programs. With waves, 18 of them can, among them single-kernel programs such as `2dconv` and `3dconv`. No GraphBIG program can, because GraphBIG launches its kernels with few blocks of 1,024 threads, so each kernel is one wave that touches nearly all of the program's pages.

7 Related work

Static footprints of thread blocks. CODA [4] inspects the index computations of a kernel and computes the stride between the data of consecutive thread blocks at launch, to place each block's data in the memory of the GPU that runs it. LADM [3] extends this index analysis to hierarchical access patterns and jointly places data and schedules thread blocks on systems of several GPUs. Unlike our footprints, both describe the accesses of a thread block by a stride, and do not determine accesses whose indices are loaded from memory.

Footprints for Unified Memory. SUV [2] uses compiler-inferred access information to place data structures in host or device memory and to prefetch at kernel boundaries, at the granularity of data structures rather than bytes. MSched [6] predicts the pages a kernel will touch from its launch arguments, from the history of earlier runs, and migrates them ahead of time when several programs share a GPU. ETC [5] manages memory oversubscription without footprints, by overlapping eviction with execution and by reducing thrashing, and finds that irregular programs suffer most from it.

Dependences between operations. Wireframe [1] executes a graph of dependences between thread blocks on the GPU. Footprints of the kind we use here are also what determines the dependences between GPU operations, which MemPrint computes from the same interpreter runs.

8 Limitations

A limitation of the unwritten units is that the footprints do not record which host buffer a copy to the host goes to, which means that the savings on copies to the host assume the copy returns each unit to the buffer it last came from or went to. A limitation of the traced footprints as an oracle is that they come from one GPU run, which means that a footprint can be unsafe for another run of a racing program even where it is safe against this one. A limitation of the footprint we rebuild for runs made before this work is that a copy that only another order's extra rounds issue is not in the per-block references, which means that it is left out of that footprint. The footprint the interpreter now writes during its run includes such a copy. A limitation of the copy model is that it charges each needed range a fixed cost and does not model copies of strided ranges, which means that it may undercount the savings on programs whose unneeded units are scattered.

A limitation of the Unified Memory model is that its fault cost and batch size are not measured on the A30, which means that the gains of prefetching, which come in part from faults avoided, are shown over a range of fault costs rather than for one measured value. A limitation of the waves is that they issue a kernel's blocks in fixed groups of equal duration, while the GPU starts a block whenever another finishes, which means that the pages in use at once may differ from a wave's. A limitation of the footprints of blocks is that they do not order the accesses within a block, which means that a program whose kernels each run one wave, such as every GraphBIG program on the large inputs, cannot be modelled in less memory than its data. The model also runs every operation after the previous one, so a prefetch does not overlap the kernel before it, as `cudaMemPrefetchAsync` on another stream could.

9 Conclusion

Exact footprints of GPU operations show that 22.1% of the bytes the benchmarks copy to the device on the small inputs, and 4.5% on the large ones, are never read, and leaving them out saves up to 78.3% of the GPU operation time of one program but little in most. The interpreter's footprints are safe to act on only when taken over every thread order, at a cost of part of the savings in programs whose writes depend on loaded values. On Unified Memory with less device memory than the program's data, the footprints of each wave of thread blocks let a runtime system prefetch and evict pages as well as an oracle would, which gains 1.407× over demand paging, where prefetching whole buffers gains little. We leave for future work measuring these gains on the GPU, and ordering the accesses within a block, which programs whose kernels run as one wave would need.

References

[1] AmirAli Abdolrashidi, Devashree Tripathy, Mehmet Esat Belviranli, Laxmi Narayan Bhuyan, and Daniel Wong. Wireframe: Supporting data-dependent parallelism through dependency graph execution in GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 600–611, 2017.

[2] Pratheek B, Guilherme Cox, Ján Veselý, and Arkaprava Basu. SUV: Static analysis guided unified virtual memory. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 293–308, 2024.

[3] Mahmoud Khairy, Vadim Nikiforov, David Nellans, and Timothy G. Rogers. Locality-centric data and threadblock management for massive GPUs. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1022–1036, 2020.

[4] Hyojong Kim, Ramyad Hadidi, Lifeng Nai, Hyesoon Kim, Nuwan Jayasena, Yasuko Eckert, Onur Kayiran, and Gabriel H. Loh. CODA: Enabling co-location of computation and data for multiple GPU systems. *ACM Transactions on Architecture and Code Optimization*, 15(3):32:1–32:23, 2018.

[5] Chen Li, Rachata Ausavarungnirun, Christopher J. Rossbach, Youtao Zhang, Onur Mutlu, Yang Guo, and Jun Yang. A framework for memory oversubscription management in graphics processing units. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 49–63, 2019.

[6] Weihang Shen, Yinqiu Chen, Rong Chen, and Haibo Chen. Towards fully-fledged GPU multi-tasking via proactive memory scheduling. arXiv:2512.24637, 2025.

[7] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 372–383, 2019.